Git and GitHub In-depth

Research Computing
UNIVERSITY OF COLORADO BOULDER

Be Boulder.

Working with Git and GitHub

John Reiland

- Research Computing
- Website: www.colorado.edu/rc
- Helpdesk: rc-help@colorado.edu

Matthew Murray

- Center for Research Data & Digital Scholarship (University Libraries)
- Website: colorado.edu/crdds/
- Helpdesk: crdds@colorado.edu

Slides:
https://github.com/ResearchComputing/git_github_in_depth_short_course
Survey: http://tinyurl.com/curc-survey18

Goals

- Convince you that basic Git/GitHub fluency is:
  - Easy
  - Practical
  - An extremely important tool in your tool belt!

Learning Goals

- Know the differences between Git and GitHub
- Understand the basics of version control
- Learn basic Git and GitHub fluency
  - Creating a repo, add, commit, pull, clone, push
- Collaboration in GitHub

Outline

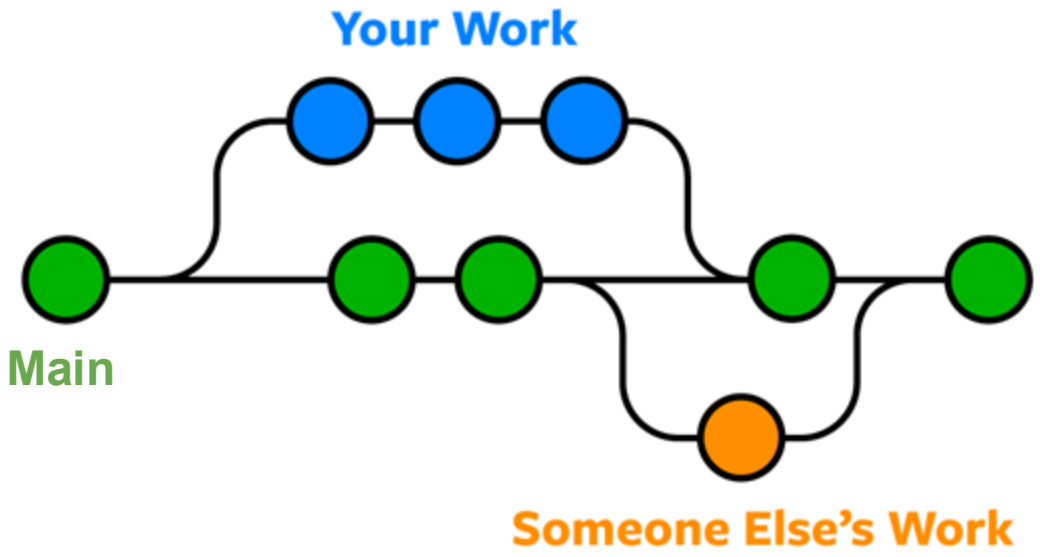
- What is version control?
- Brief overview of Git and GitHub
- When not to use GitHub
- Creating your own repository locally
- Pushing local changes to GitHub
- Documentation

What is version control?

- Version control is the practice of tracking and managing changes to files.
  - Revert to various states of files
  - Modify items without harming the original copy
- Not limited to code
  - Can be used for documents, images, etc.

Benefits of version control

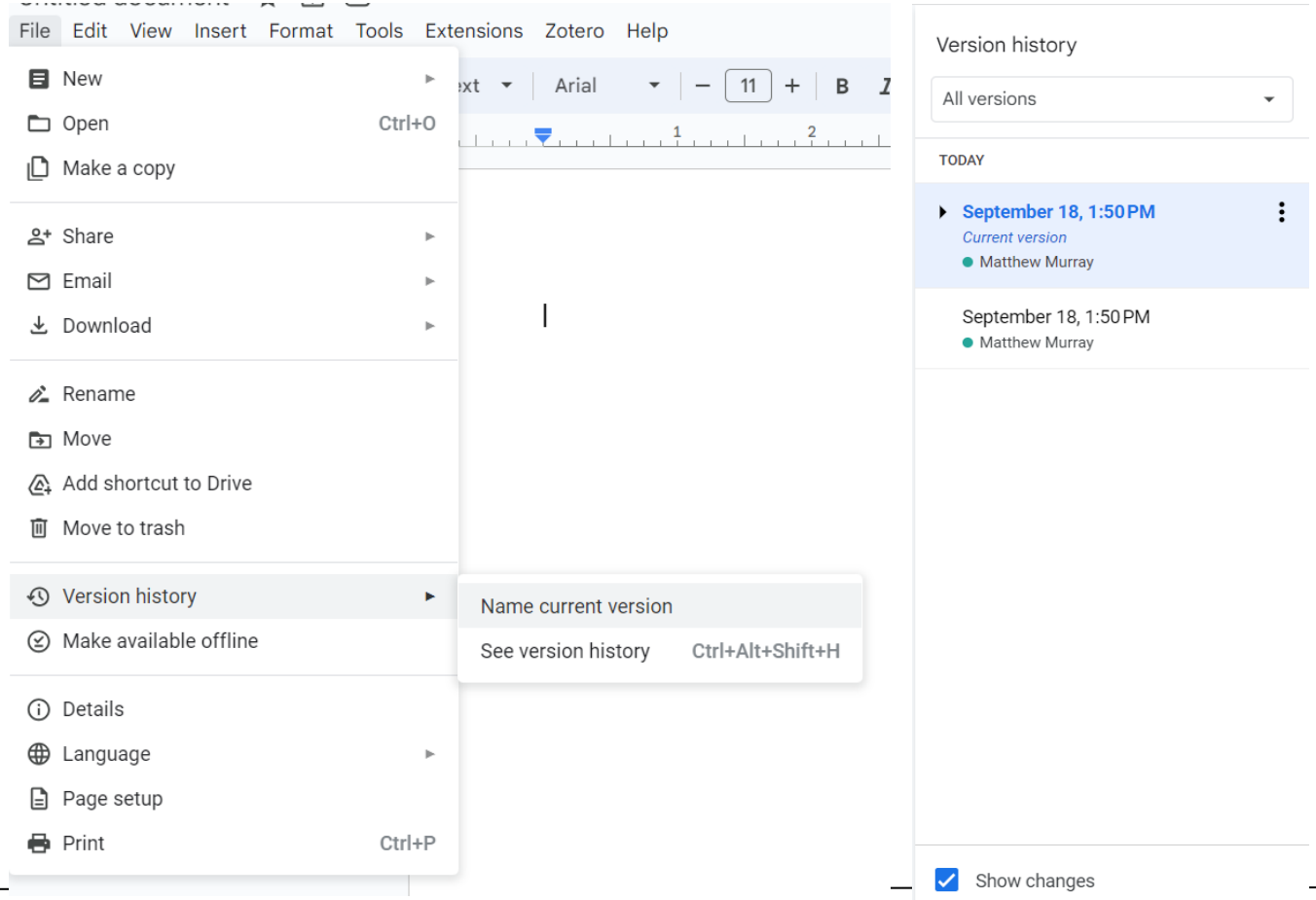
- Using version control provides
  - Clear tracking of the project's history
  - Management and view of different branches
  - Collaboration through merging of branches



Images: nobledesktop.com

What is version control?

- Google Docs includes "Version History"
- This allows you to see what changes were made, when those changes happened, and who made them
- You can also revert to a previous version of your file



What is Git?

- Git is version control software, created by Linus Torvalds, the same person who created the Linux operating system.
- Git monitors files on your computer and tracks changes made to them over time
- Uses the command line

Why is it called “Git”?

- Linus Torvalds: "I'm an egotistical bastard, and I name all my projects after myself. First 'Linux', now 'Git’”.
- Git: “A silly, incompetent, stupid or annoying person (usually a man).” (Wikitionary)

git / README

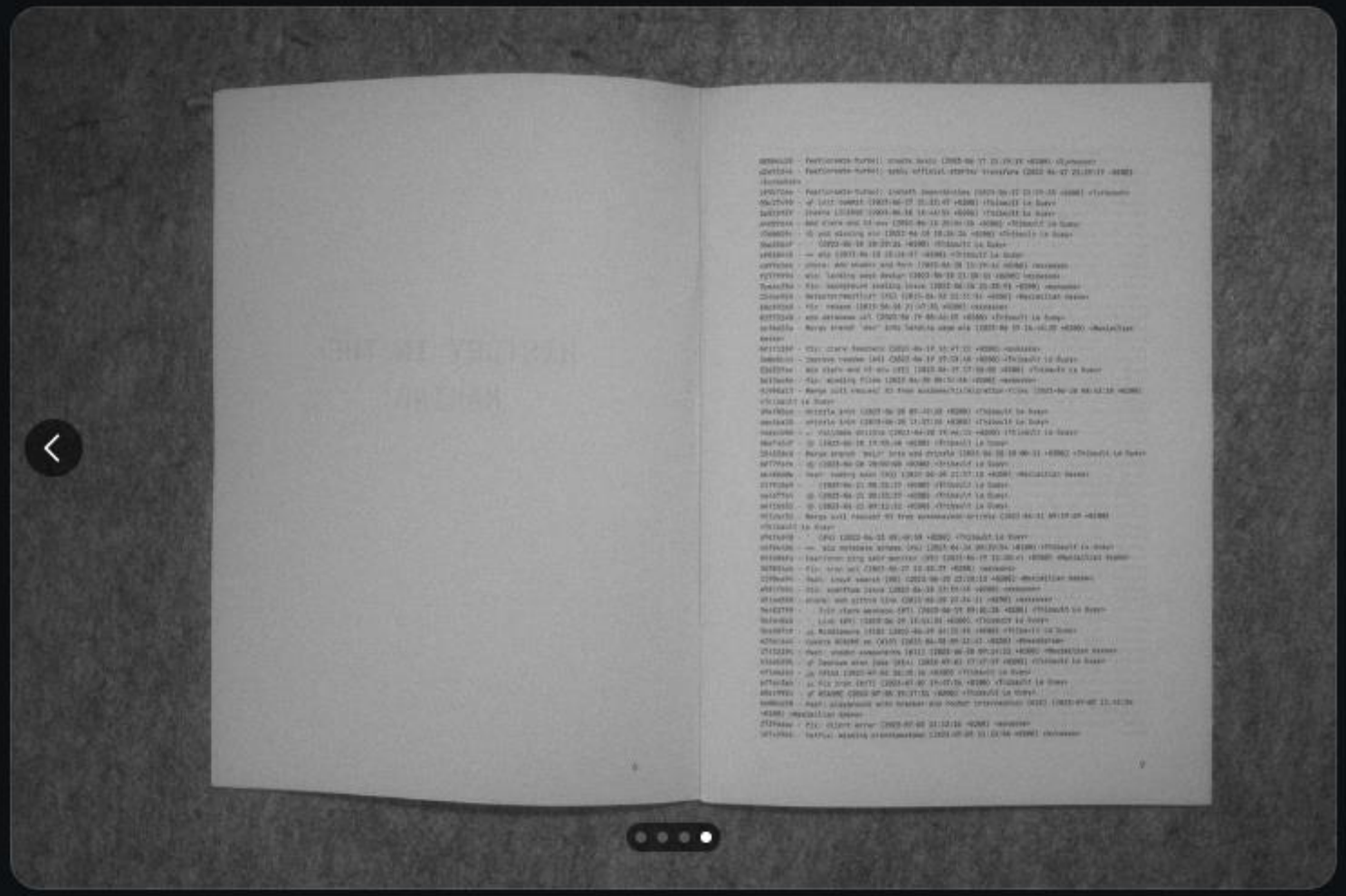
Linus Torvalds Initial revision of "git", the information manager from hell

Code Blame 168 lines (135 loc) · 8.2 KB

```
1
2            GIT - the stupid content tracker
3
4    "git" can mean anything, depending on your mood.
5
6     - random three-letter combination that is pronounceable, and not
7       actually used by any common UNIX command.  The fact that it is a
8       mispronounciation of "get" may or may not be relevant.
9     - stupid. contemptible and despicable. simple. Take your pick from the
10      dictionary of slang.
11    - "global information tracker": you're in a good mood, and it actually
12      works for you. Angels sing, and a light suddenly fills the room.
13    - "goddamn idiotic truckload of sh*t": when it breaks
14
```

r/zines • 5 days ago
tibozaurus

We made a zine from our Git history



We are an open-source project, so we decided to print our Git history to celebrate our 2nd anniversary.

https://www.reddit.com/r/zines/comments/1miy3bs/we_made_a_zine_from_our_git_history/

What is GitHub?

- GitHub is a Microsoft subsidiary that offers cloud hosting for Git repositories
- Provides a GUI (Graphical User Interface) for Git
  - Most (but not all) features are available in the GitHub Desktop client
- Allows for easy collaboration and sharing
  - Issue queues for bugs and features, pull requests, and more
- GitHub basic is free (has limits)
  - Hosts both open and private repositories
- GitHub Enterprise (free for CU affiliates)
  - Includes cloud-based development environments
  - https://oit.colorado.edu/services/business-services/github-enterprise

Git vs GitHub

- Git: version control system
  - the actual software

- GitHub
  - cloud-based storage website

What Can I Use Git and GitHub For?

Git

- Track and manage changes of files
- Document changes
- Collaborate with others

GitHub

- Off-site backup for Git repositories
- Contribute to Open Source projects
- Communicate with team members
- Communicate with your users
- Join communities
- Bug tracking
- Website hosting

Alternatives to…

- Git (version control)
  - Apache Subversion (SVN)
  - Mercurial SCM
  - CVS (Concurrent Versions System)
- GitHub (hosting)
  - Codeberg
  - GitLab
  - Forgejo
  - Bitbucket

When NOT to use GitHub

- When you are looking for long-term preservation
  - There's no guarantee Microsoft will keep GitHub around forever
  - Lots of source-code-hosting platforms no longer exist
  - Thousands of URLs in research articles no longer work as they point to code hosted by defunct services
- When you want your code to be cited
  - Zenodo is an Open Access repository that can be linked to GitHub
  - Allows you to archive a specific release of public GitHub projects
  - Creates a DOI for the archive that you can use in citations

zenodo

Search

Upload Communities

themattthewmurray@gmail.com

Zenodo.org will be unavailable for 2 hours on September 29th from 06:00-08:00 UTC. See announcement.

September 2, 2023

Software Open Access

PyGMT: A Python interface for the Generic Mapping Tools

Tian, Dongdong; Uieda, Leonardo; Leong, Wei Ji; Schlitzer, William; Fröhlich, Yvonne; Grund, Michael; Jones, Max; Toney, Liam; Yao, Jiayuan; Magen, Yohai; Jing-Hui, Tong; Materna, Kathryn; Belem, Andre; Newton, Tyler; Anant, Abhishek; Ziebarth, Malte; Quinn, Jamie; Wessel, Paul

PyGMT is a library for processing geospatial and geophysical data and making publication quality maps and figures. It provides a Pythonic interface for the Generic Mapping Tools (GMT), a command-line program widely used in the Earth Sciences.

The development of PyGMT has been supported by NSF grants OCE-1558403 and EAR-1948603.

Preview

baseline-images.zip

baseline-images

test_basemap.png	6.2 kB
test_basemap_compass.png	71.2 kB
test_basemap_loglog.png	24.5 kB
test_basemap_map_scale.png	31.0 kB
test_basemap_polar.png	31.9 kB

18,263 views

1,446 downloads

See more details...

Available in

GitHub

Indexed in

OpenAIRE

Publication date:

zenodo

Search

Upload Communities

thematthewmurray@gmail.com

Zenodo.org will be unavailable for 2 hours on Septembe

September 2, 2023

PyGMT: A Python inte Mapping Tools

Tian, Dongdong; Uieda, Leonardo; Leong, Wei Ji
Jones, Max; Toney, Liam; Yao, Jiayuan; Magen,
Newton, Tyler; Anant, Abhishek; Ziebarth, Malte;

PyGMT is a library for processing geospatial and geophy provides a Pythonic interface for the Generic Mapping T Sciences.

The development of PyGMT has been supported by N

Preview

baseline-images.zip

- baseline-images
  - test_basemap.png
  - test_basemap_compass.png
  - test_basemap_loglog.png
  - test_basemap_map_scale.png
  - test_basemap_polar.png

24.5 kB
31.0 kB
31.9 kB

Versions

Version v0.10.0 10.5281/zenodo.8303186	Sep 2, 2023
Version v0.9.0 10.5281/zenodo.7772533	Mar 31, 2023
Version v0.8.0 10.5281/zenodo.7481934	Dec 30, 2022
Version v0.7.0 10.5281/zenodo.6702566	Jul 1, 2022
Version v0.6.1 10.5281/zenodo.6426493	Apr 11, 2022

View all 17 versions

Cite all versions? You can cite all versions by using the DOI 10.5281/zenodo.3781524. This DOI represents all versions, and will always resolve to the latest one. Read more.

18,263 views
1,446 downloads
See more details...

vailable in
GitHub

dexed in
OpenAIRE

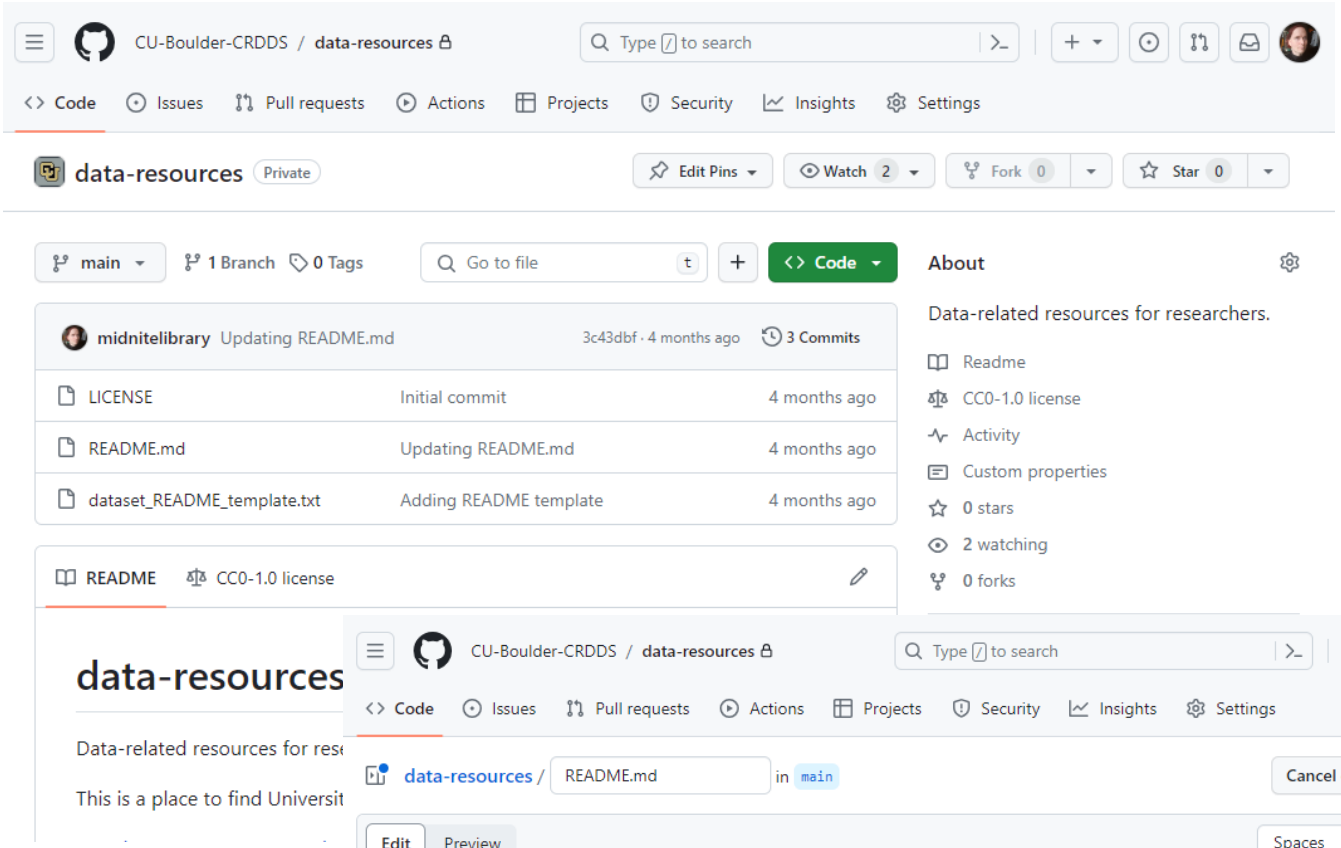
Publication date:

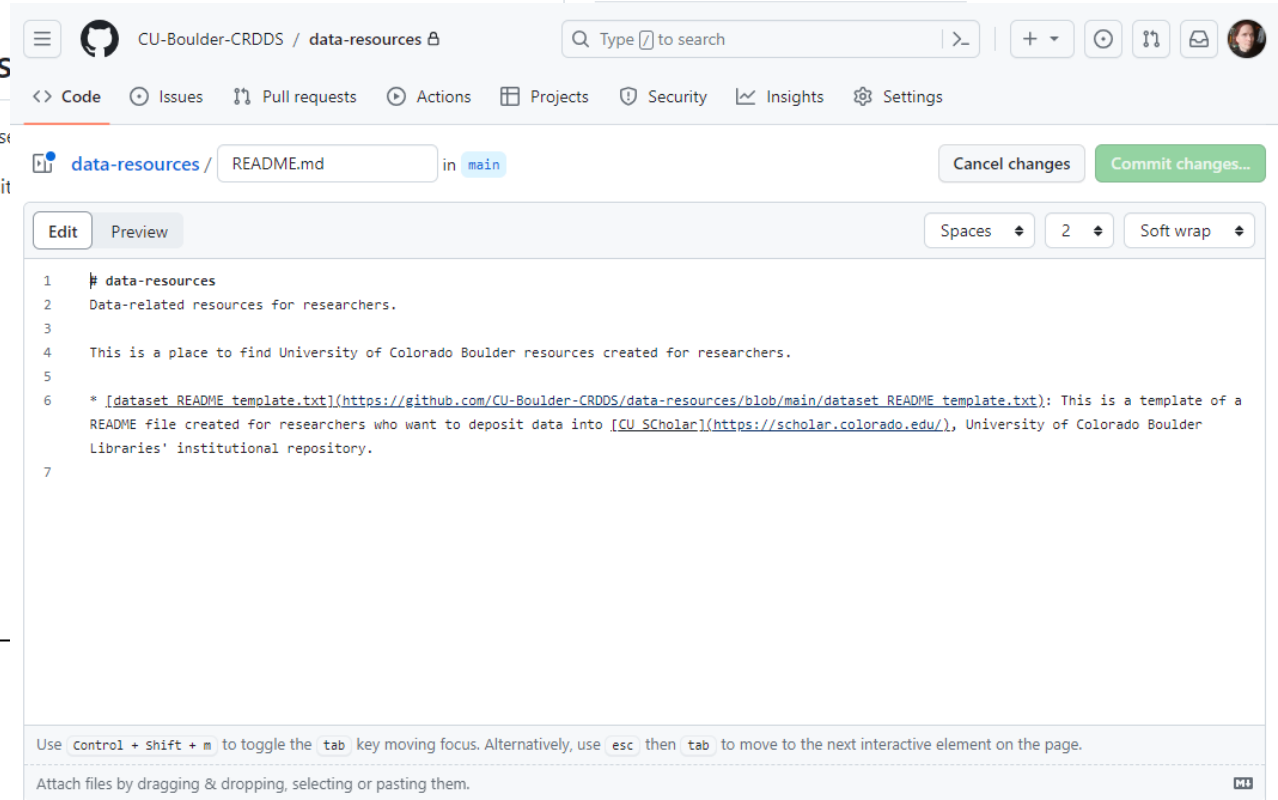
Ways to interact with GitHub

- The website
- GitHub Desktop
- Using Git and the command line

Website

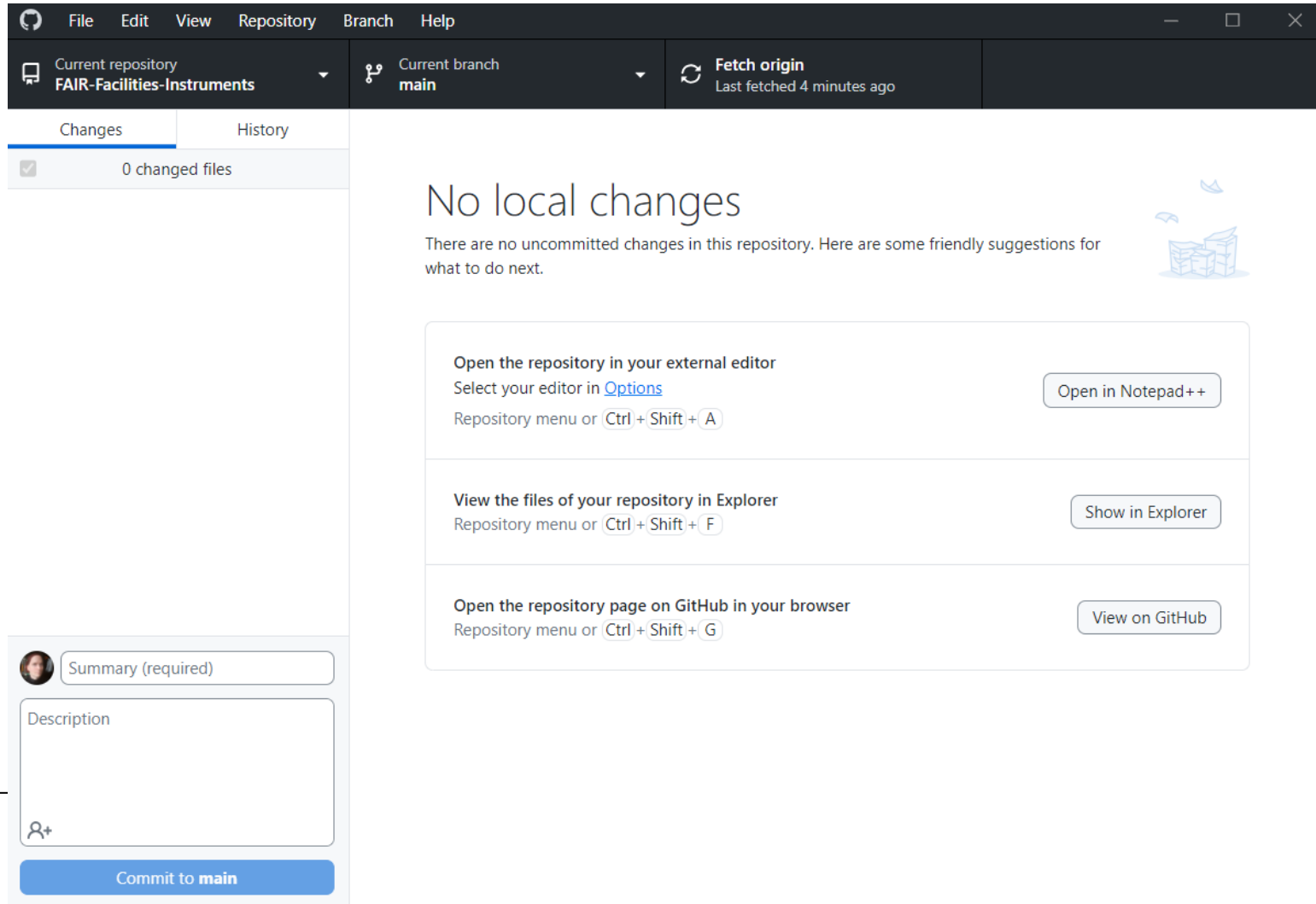
- Create and manage projects
- Upload and download files
- Write documentation





GitHub Desktop

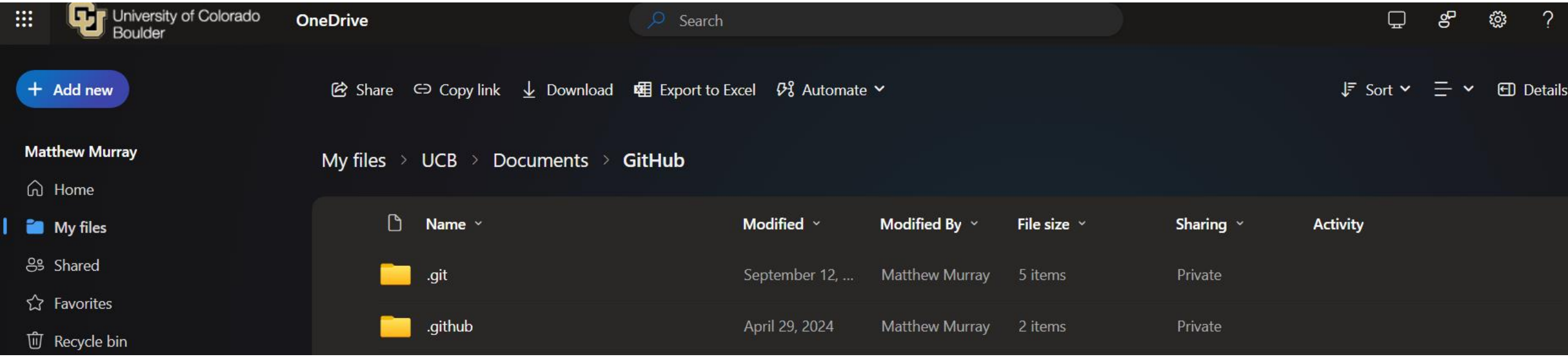
- Create, clone, and fork projects
- Commit changes and submit contributions
- Many people who use Git frequently prefer to use the command line
- Other graphical Git interfaces exist



The Command Line

- The most powerful and, for many, the best way to really learn the ins and outs of Git.
- Has the steepest learning curve. The command line interface requires you to learn and memorize all the available commands, the workflow, etc.
  - Cheat sheets are available!
- We'll be talking about this more during this session!

Be careful where you put your GitHub directory



- Microsoft OneDrive uploads a lot of stuff to the cloud, which you don't always want

Getting Started with Git

Definitions

- Trunk/Master: Main version of a repository
- Clone: A copy of a repository that is stored locally instead of online
- Branch: A parallel version of a repository that allows modifications to be made without affecting the main file
- Fork: A branch that is not intended to be merged back into the main file
- Pull/Fetch: Getting the latest version of a repository
- Merge: Combining changes from different branches/versions of files
- Commit: A saved snapshot of your project and any changes you have made
- Push: Sending your local changes to an online repository

Things to Remember about the Git Workflow

- Git isn’t an automatic process (it's not like autosave)
- In order to use it, you need to integrate Git into your workflow
- You set it up for a project, on a directory on your hard drive, and from that point forward Git will start watching for any changes in that directory.
- You do your work as normal, using all the same software and tools you’re used to using
- At any point, you can ask Git to tell you what changes it has detected and it will show you a detailed report of every addition and deletion that it has detected
- At any point you can make a commit and create a snapshot of your project

Getting Started

- A GitHub account set up.
  - If you don't have one, sign up at GitHub.com.

- Many systems have Git installed; however, you may need to download it on your local machine:
  - See https://git-scm.com/book/en/v2/Getting-Started-Installing-Git for more information on installing Git
  - To check if Git is installed, run:

```
$ git -version
```

Linking Git to GitHub

- Once downloaded, we can then configure Git with our GitHub username and email via the command line. This allows us to interact with our GitHub more easily.
  - First, set your username. For example, if my GitHub username is gh-user, then I would do the following:

    ```
    $ git config --global user.name “gh-user”
    ```

  - Now, set your email. For example, if my email for GitHub is gh-user@gmail.com, then I would do the following:

    ```
    $ git config --global user.email “gh-user@gmail.com”
    ```

- Confirm Git has been configured (should show your entered info)

```
$ git config --list
```

Personal Access Token

- Allows you to verify your identity with GitHub
  - For more information, see https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens
- Downside is that you need to enter your username and then the Personal Access Token as your password for events such as:
  - Any interaction with a private repo
  - Pushing to a public repo
- There are ways to store your username and token, but these require third-party software

SSH Keys

- Alternative way to verify your identity with GitHub
  - For more information see: https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent
  - Be sure to select the proper Operating System when using the link
- Setup is more involved, but makes it so that you never have to enter your username and token when interacting with a private repo or pushing to a public repo

Getting Started with Git (local)

Hands on tutorial

Goal: Create a simple project that contains a markdown file

First let's create a new directory for our project:

```
$ mkdir git_work
$ cd git_work
$ mkdir git-tutorial
$ cd git-tutotial
```

Git Repository (Repo)

- A Git repository tracks and saves the history of all changes made.
  - All this information is stored in “.git”, which is the repository folder

- We can make a directory (folder) a Git repo using “git init”

Git Init

- In your “git-tutorial” directory run

```
$ git init
```

- Git creates the "hidden" directory called “.git”

```
$ ls -la
```

- Your directory is now a repo!
  - Git is now ready to to be used
  - Allows us to tell Git what items to watch

Create the main branch

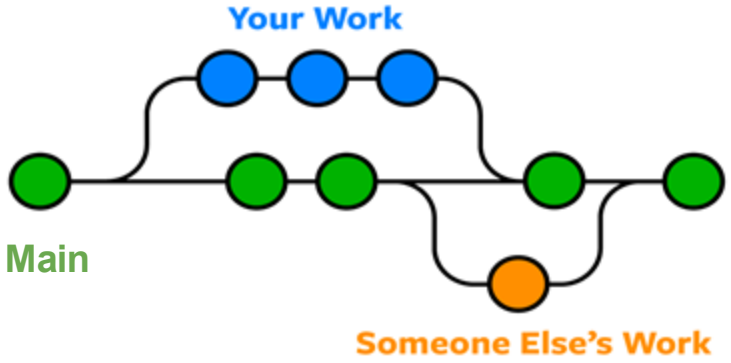
Now that we have a repo, we can create branches. Branches are a version of the repository.

- It is customary to name the primary branch “main”
- This can be done as follows (after an init)

```
$ git checkout -b main
```

- You can switch between branches

```
$ git checkout <branch-name>
```



Let’s add a file!

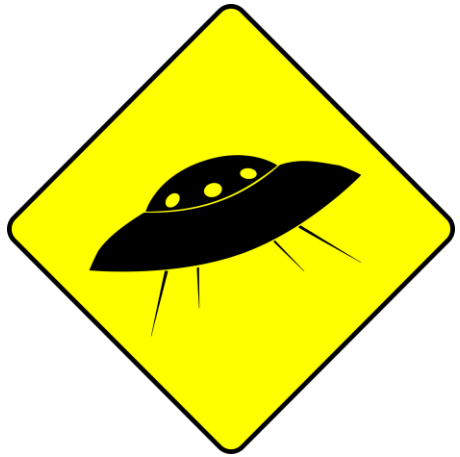
- It is customary to add a README.md
  - Description of repo and any helpful information

- To add a README.md, in “git-tutorial” create and edit the file using nano (or an editor of your choice)

```
$ nano README.md
```

  - Add anything you would like!
  - Be sure to save the file when you exit.

Git does not know about README.md yet!!



Areas of Git Workflow

Working Area

- Items that you are currently working on
- Are not tracked by Git!
- Exists locally
- Created by using “git init”

Staging Area

- When Git starts tracking and saving your work
- Exists locally
- Items are added to this area by using “git add”

Snapshot Area

- All staged items are captured
- Version of the repo
- Exists locally
- Items are added to this area by using “git commit”

GitHub

- Exists locally and on GitHub!
- Items are added to this area using “git push”

Git Status

- The git status command displays the state of the working and staging area.

- Let's see what area README.md is in

```
$ git status
```

  - We see it is an untracked file, so it is in the working area

What if you don’t want Git to track something?

.gitignore

We can add a file named ".gitignore" to our repo

- Specifies what items (files, directories, etc.) should never be tracked

- Let's create a file to ignore!

```
$ echo "Super secret stuff" > confidential_data.txt
```

- Add ".gitignore" to "git-tutorial" and put "confidential_data.txt" in it

```
$ echo confidential_data.txt > .gitignore
```

Let’s add our files to the staging area now!

Git Add

- The git add command adds a change in the **working area to the staging area.**

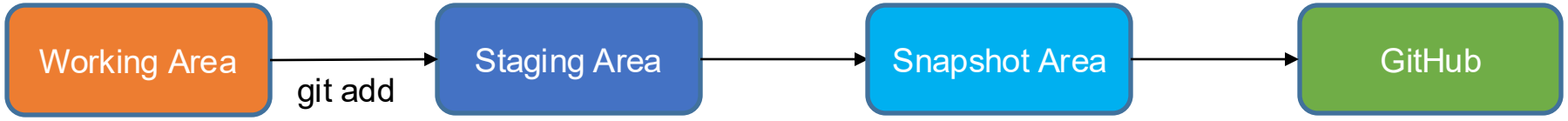
- Let's add our README.md to the staging area

```
$ git add README.md
```

or add everything in the current directory

```
$ git add .
```

- Anytime a change is made, you need to do a git add (to track them)



Git Reset

- If you've already staged files using git add and you want to move them back to the working directory, you can use the following command:

```
$ git reset <file>
```

- To unstage the README.md file:

```
$ git reset README.md
```

Git Commit

- The git commit command captures a snapshot of all staged items
  - Commits can be thought of as a version of the repo
  - Commits should be accompanied with a brief message

- Let's commit our staged item!

  $ git commit -m "Create repo, add README.md, add .gitignore"

  $ git status

Common practice – add, commit

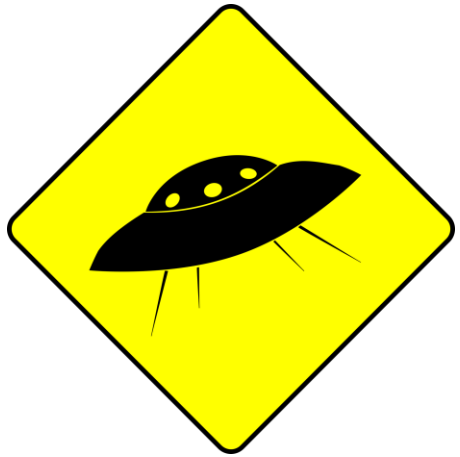
- git add
  - Can be performed as much as you want
  - Doesn’t need to be done after every change
- git commit
  - Always include a comment!!
  - Bundle common staged items together
  - Try not to put too many things in a commit

Git Log

- The command git log **lists the commits made in that repository**
- Lists the most recent commits first

```
$ git log
```

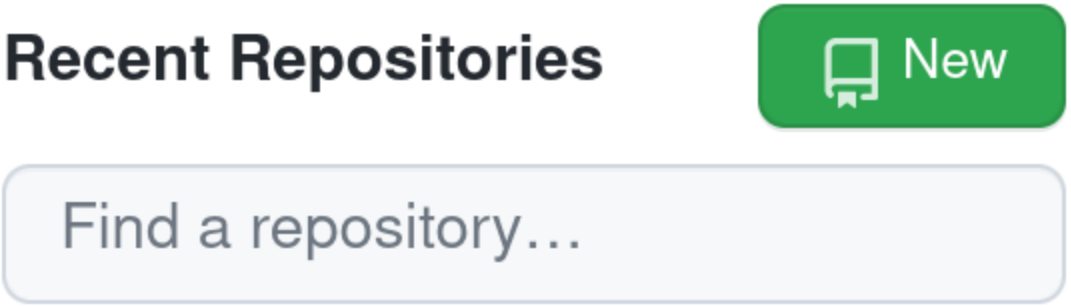
All changes and files are only locally stored right now!



To GitHub we go!

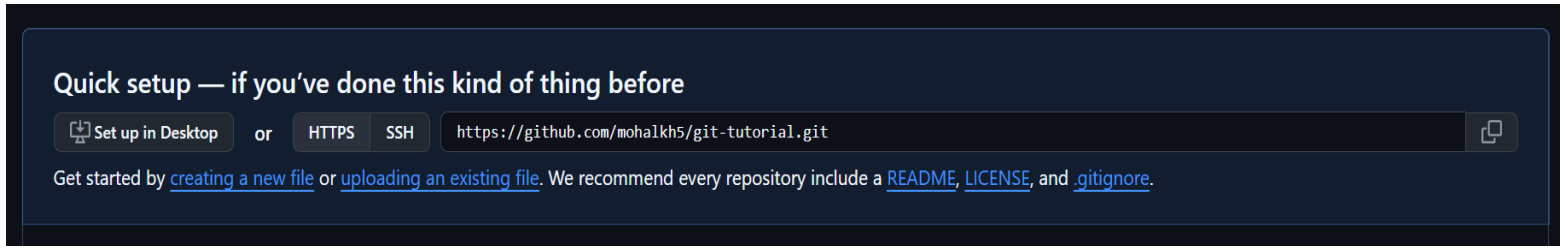
GitHub

- When you first create a repo locally, you will need to setup a new repository on GitHub too
  - Go to https://github.com
  - Sign in
  - Click on “Create New Repository” or just “New”



Create Repo in GitHub

- Name your repo, I chose “git-tutorial”
- Don’t add a README or a .gitignore
- Click “Create repository”
- We have set everything up in the previous slides, we only need to copy the ssh link!



Linking local repo to GitHub repo

Git Remote

- Used to identify the remote (e.g. GitHub) repos are linked to your local repo
- Used to link remote repos to your local repo
- To view currently linked remote repos:

```
$ git remote -v
```

- To link our remote repository:
  - When using an SSH key do:

```
$ git remote add origin git@github.com:<user>/git-tutorial.git
```

  - When using a Personal Access Token do:

```
$ git remote add origin https://github.com/<user>/git-tutorial.git
```

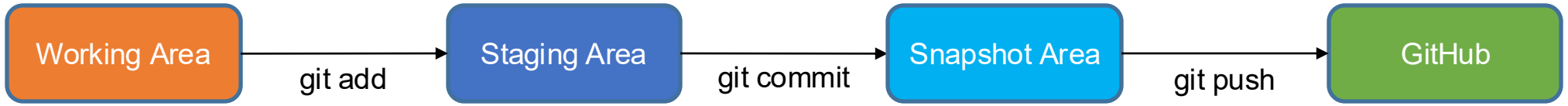
Sending local changes to GitHub

Git Push

Uploads local repository content to a remote repository

- Pushing is how you transfer commits from your local repo to a remote repo

```
$ git push <name of remote repo> <branch>
$ git push origin main
```



GitHub

- Go back to GitHub and refresh your page
  - should see the files we have added (and not the ones we've ignored)
- Some cool features!
  - look at our commits
  - directly edit/commit in the browser
- Let's do that! Let's something and commit it on GitHub
  - But now our remote repo is one commit ahead of our local one…

Git Fetch & Merge

- Git fetch retrieves the changes from the remote repo

```
$ git fetch
```

- Git merge combines two branches

```
$ git merge origin/main
```

There’s an easier way!

Git Pull

- Git pull combines the fetch and merge commands

```
$ git pull <name of remote repo> <branch>
$ git pull origin main
```

IMPORTANT!

- Make sure no commits have been done on local branch
- It is fine to have staged items (git add)
- ALWAYS do git pull before any commits!

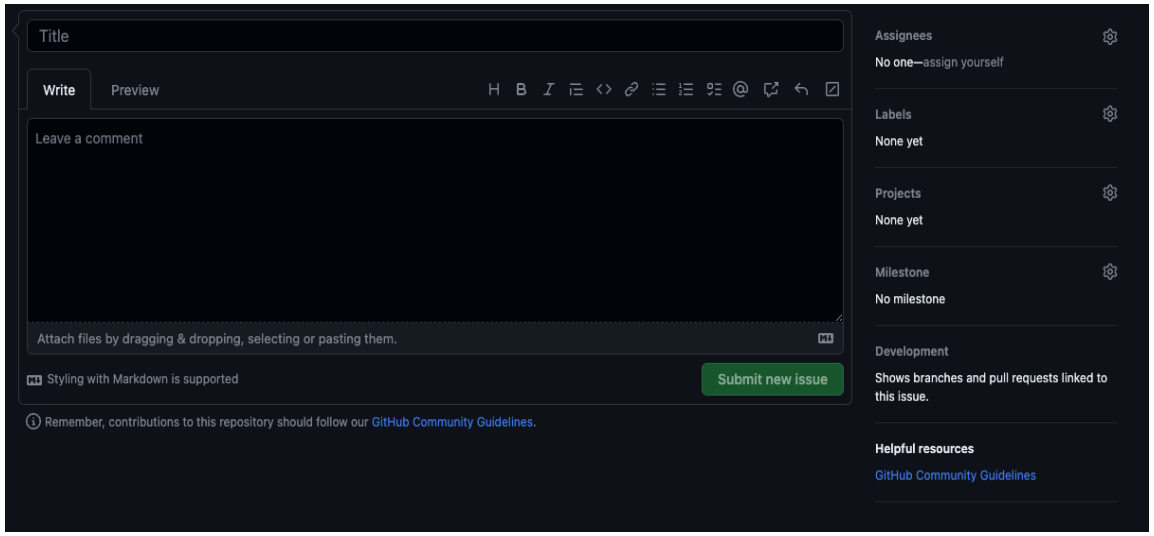
Advanced topic: Collaboration

GitHub Issues

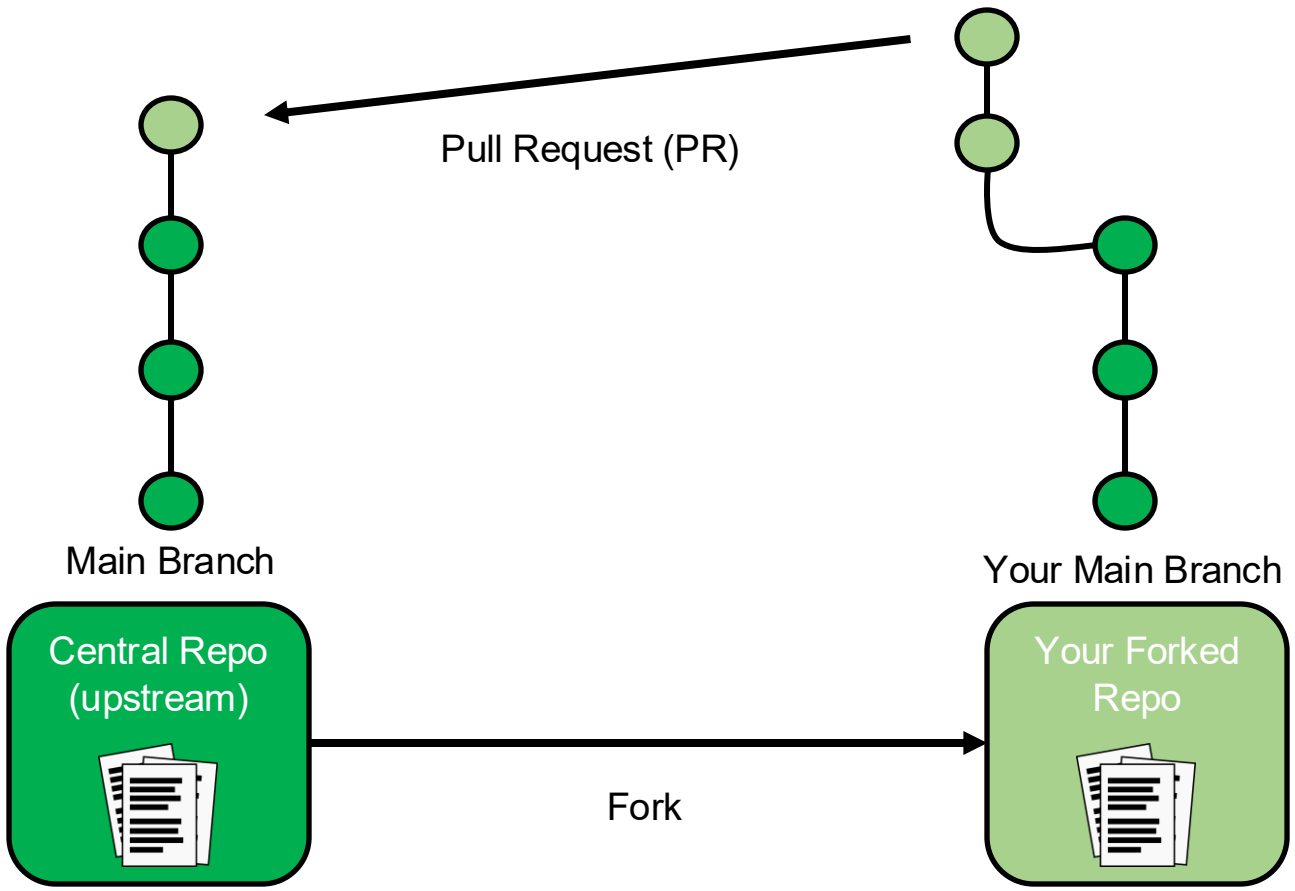
- Allows you to discuss the project
- Point out issues, request features, ask for help
- Useful place to see past user discussion

GitHub Issues

- Include as much detail as possible
  - Version of software
  - Operating system
- Provide a simple minimal example, if possible
- If a feature request
  - Outline possible implementation
  - Highlight its value

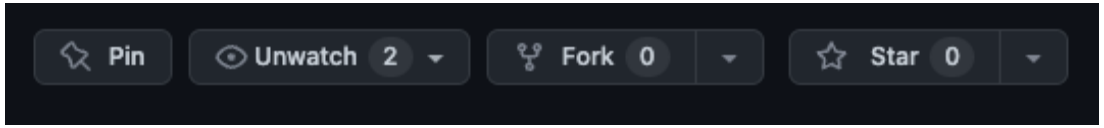


GitHub Forks



GitHub Forks

- Improves collaboration
  - Don’t have to worry about disturbing the upstream repo
  - Improves transparency through pull requests
- Go ahead and Fork my repo:
  - Go to https://github.com/rctutorial/hello-world
  - Click “Fork” button
  - Click “Create fork”
- Creates your own version of my repo under your GitHub



Git Clone

- Git clone **makes a clone (or copy) of a remote repo in a new directory, at another location**.

```
$ git clone <url> <optional new name>
```

- Easy way to grab third-party code, or pre-existing code you might need to work on

- Cloning when you have SSH keys (be sure to make "git_work_cloned"):

```
$ git clone git@github.com:rctutorial/hello-world.git
```

Pull Requests (PRs)

- A request that an upstream repo pull your branch into their branch
- Starting a PR does not automatically merge changes
  - Notifies maintainers of upstream repo
  - Allows maintainers to review your changes
    - Discussion of changes
    - Requested additional changes
- Maintainers of upstream repo merge in the changes

PR steps

1. Fork upstream repo
2. Clone the forked repo
3. Connect forked and cloned repo to upstream repo. Ex. using SSH keys:

   ```
   $ git remote add upstream git@github.com:rctutorial/hello-world.git
   ```

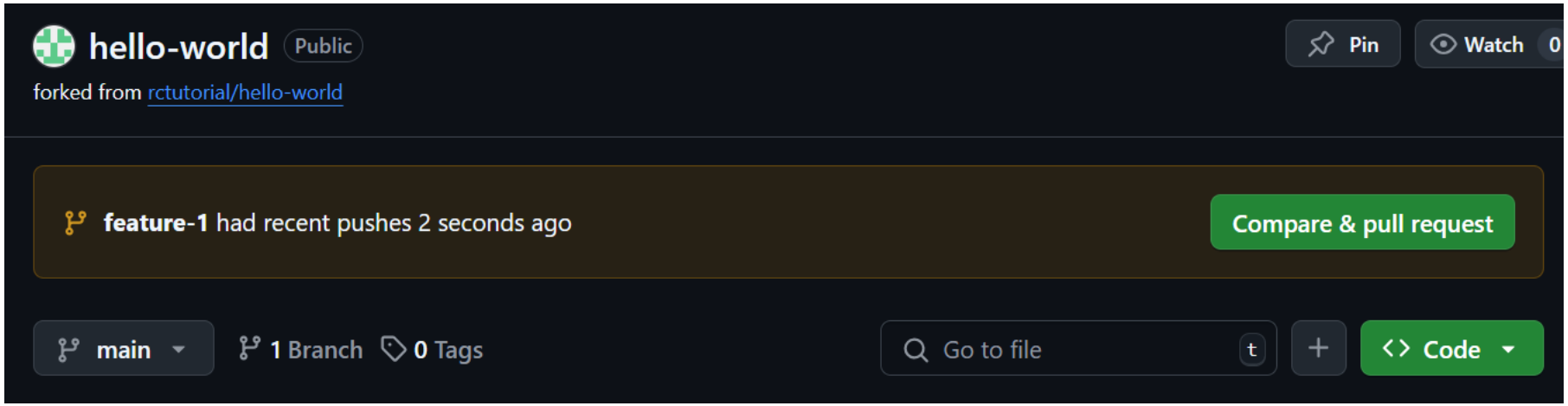
4. Create a new branch specific to your change

   ```
   $ git checkout –b <new-branch> <branch-to-copy>
   ```

5. Make your changes on this branch
6. Perform a git add, commit, and push to origin
7. Create a PR from GitHub

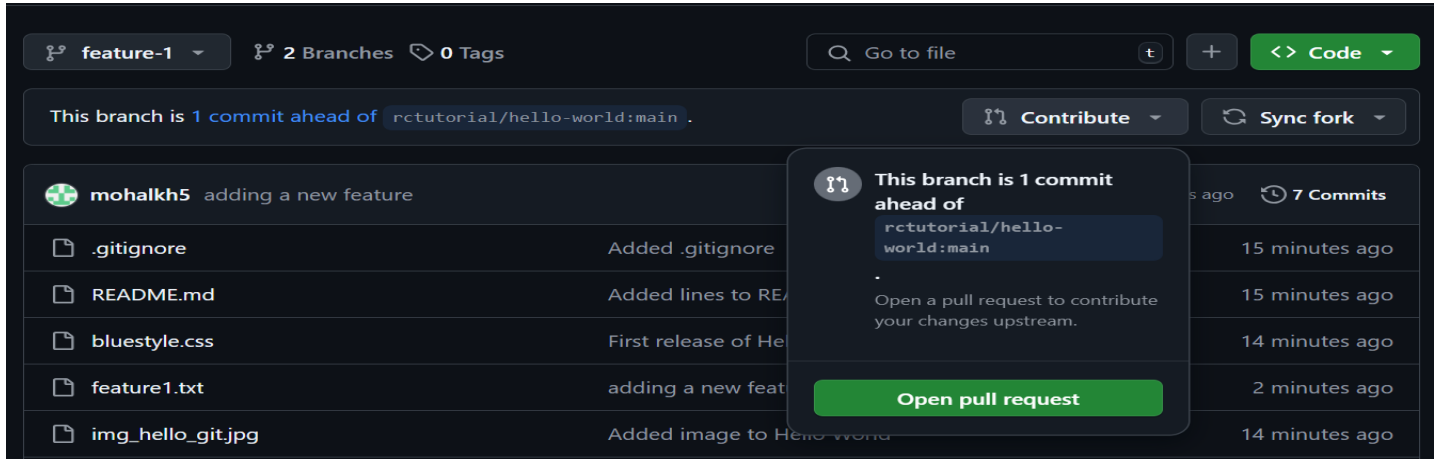
Creating a PR

- After you push your changes to the forked repo, you can click the pop-up “Compare & pull request” on GitHub
  - Will disappear after some time



Creating a PR

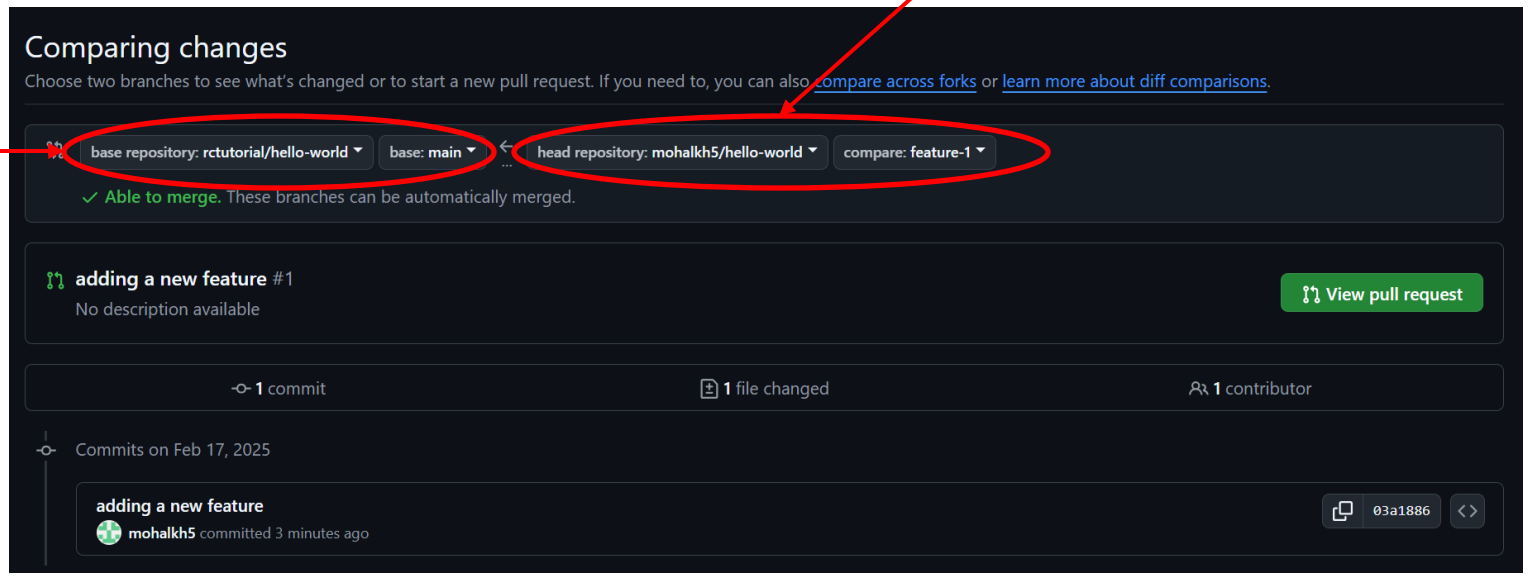
- After you push your changes to the forked repo
  1. Switch to your new branch
  2. Click the drop-down arrow next to “Contribute”
  3. Click “Open pull request”
- Will NOT disappear!



Creating a PR

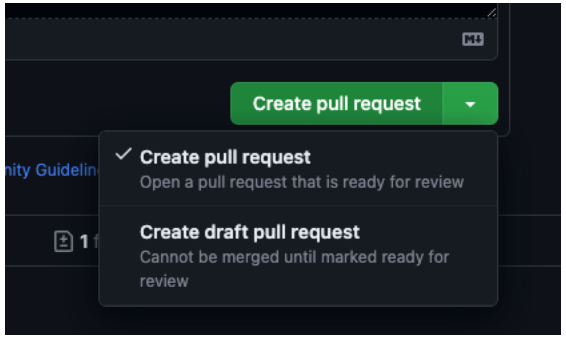
Our forked repo and branch with our changes

Upstream repo and branch we want to merge into



Pull Requests – Best Practice

- Create a new feature branch of forked repo
- When submitting a PR
  - Provide a short descriptive title
  - In comment section
    - Link to any current issue
    - Describe what the PR does and reasons for it
- Draft pull requests
  - PR is a work in progress
  - Can be used for discussion

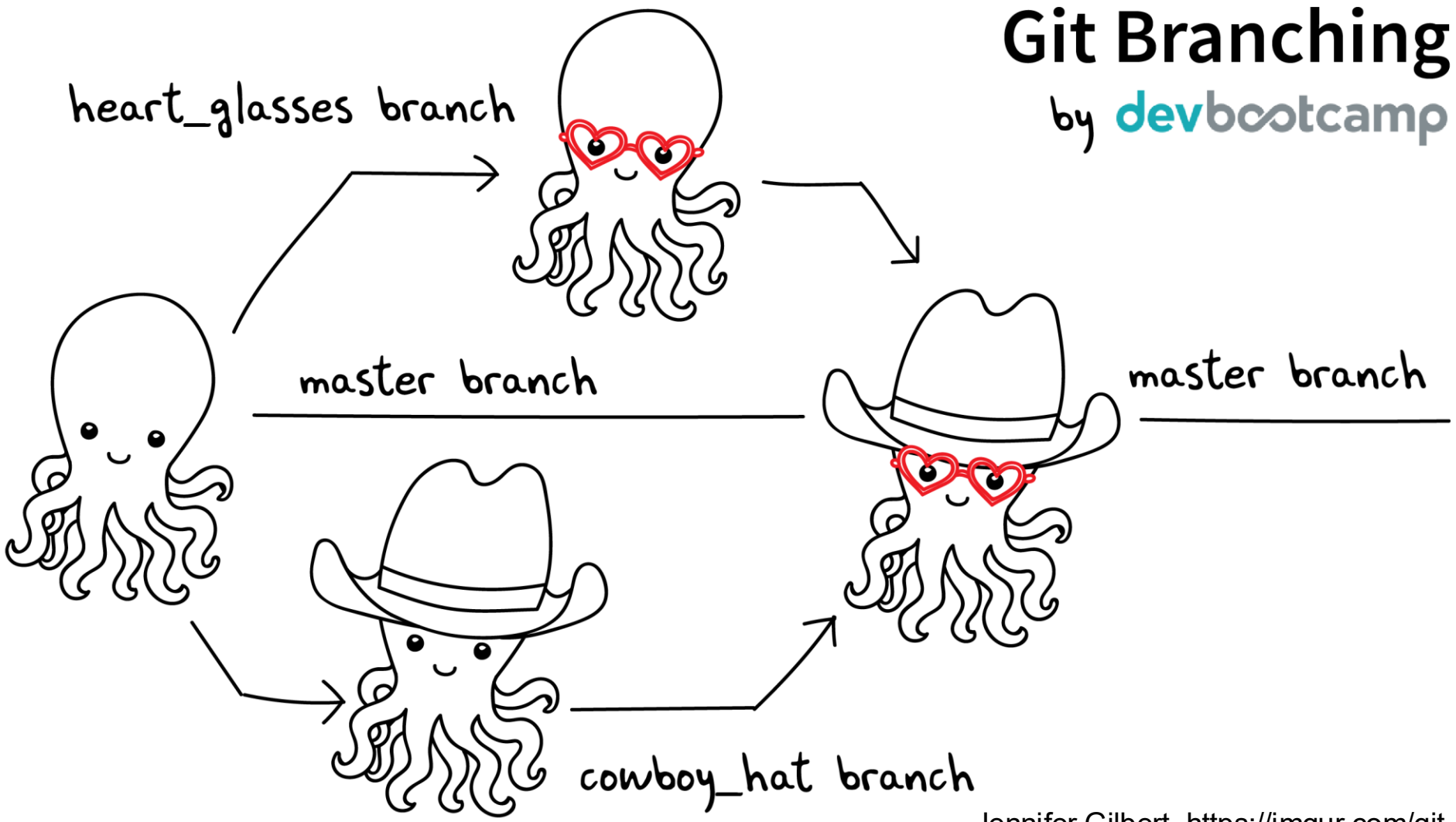


Merging

- When doing a “git pull” you are merging in changes
- This process can be done manually
- When collaborating, multiple individuals can be working on the same item
  - Conflicts can happen!
- One needs to manually resolve conflicts
- Fantastic tutorial on merging:

https://www.atlassian.com/git/tutorials/using-branches/git-merge

Branching & Merging

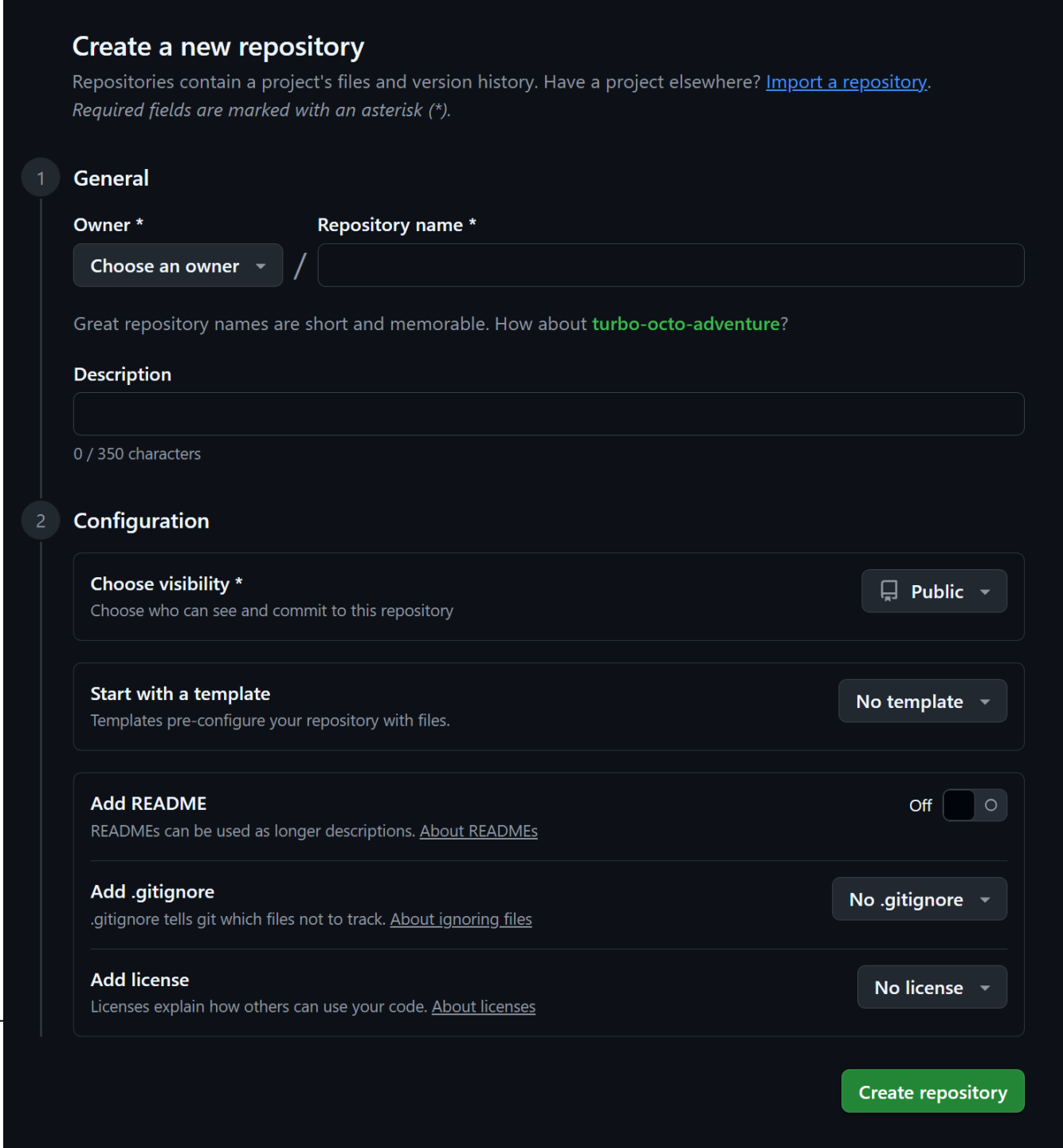


Jennifer Gilbert. https://imgur.com/git-branching-YG8In8X

Best practices when setting up a new GitHub repo

Creating a repo

- Choose an “owner”
- Add a description
- Choose visibility
- Add a README
- Add a .gitignore
- Add a license

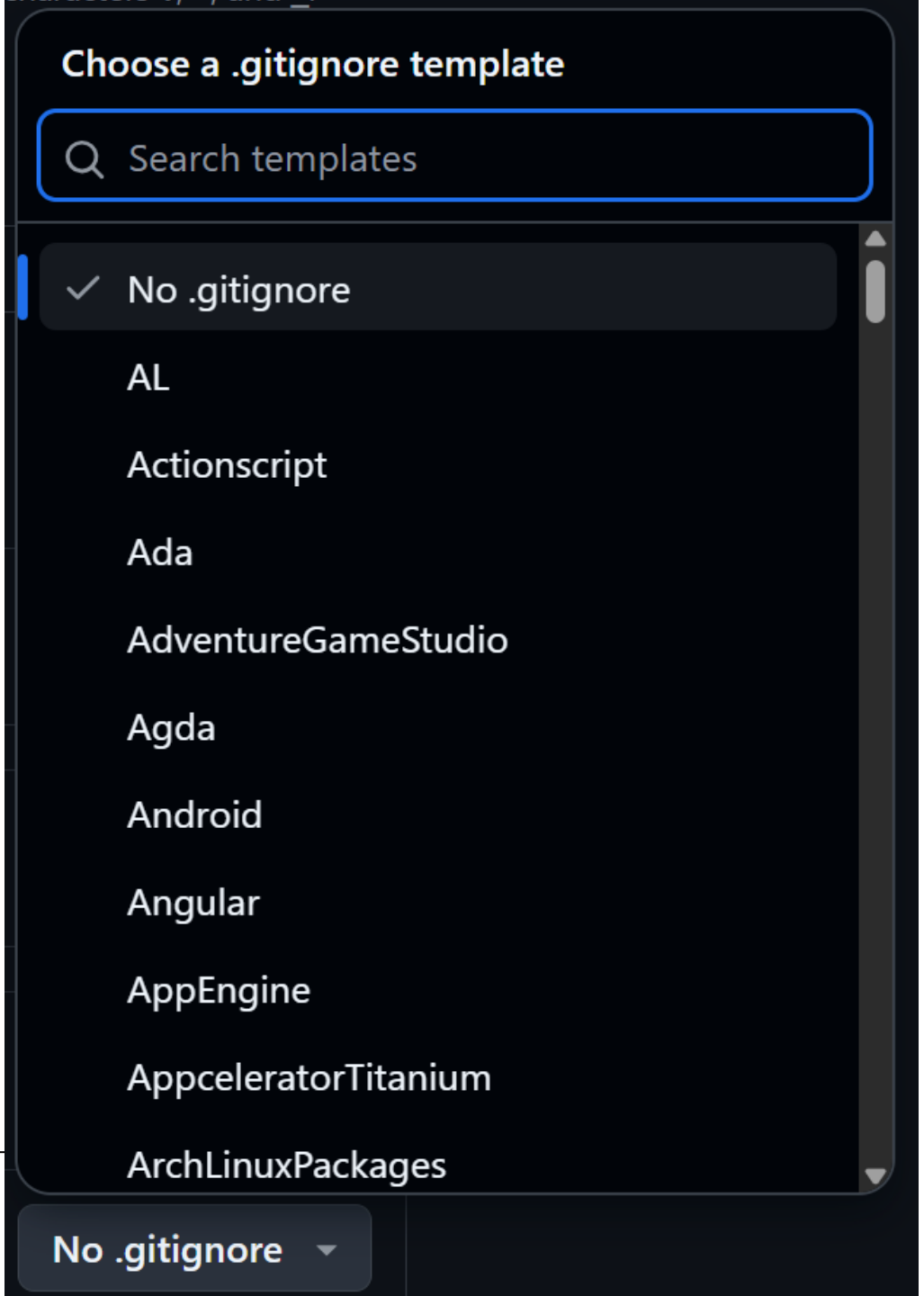


Best Practices: Documentation

- Include documentation with your project in GitHub so that others (and you) know what your project is and how it should be used.
- A README.md (markdown) file can be included in your GitHub project and will display on the front page
- What to include in a README:
  - What your project does
  - How people can use it
  - Who you are and how to contact you
  - License information
- Lots of examples and templates available
- We can provide feedback on documentation

.gitignore

- Specifies what items (files, directories, etc.) should never be tracked by Git
- Use for information that needs to be kept private or is irrelevant to a project
- Use a template or create your own



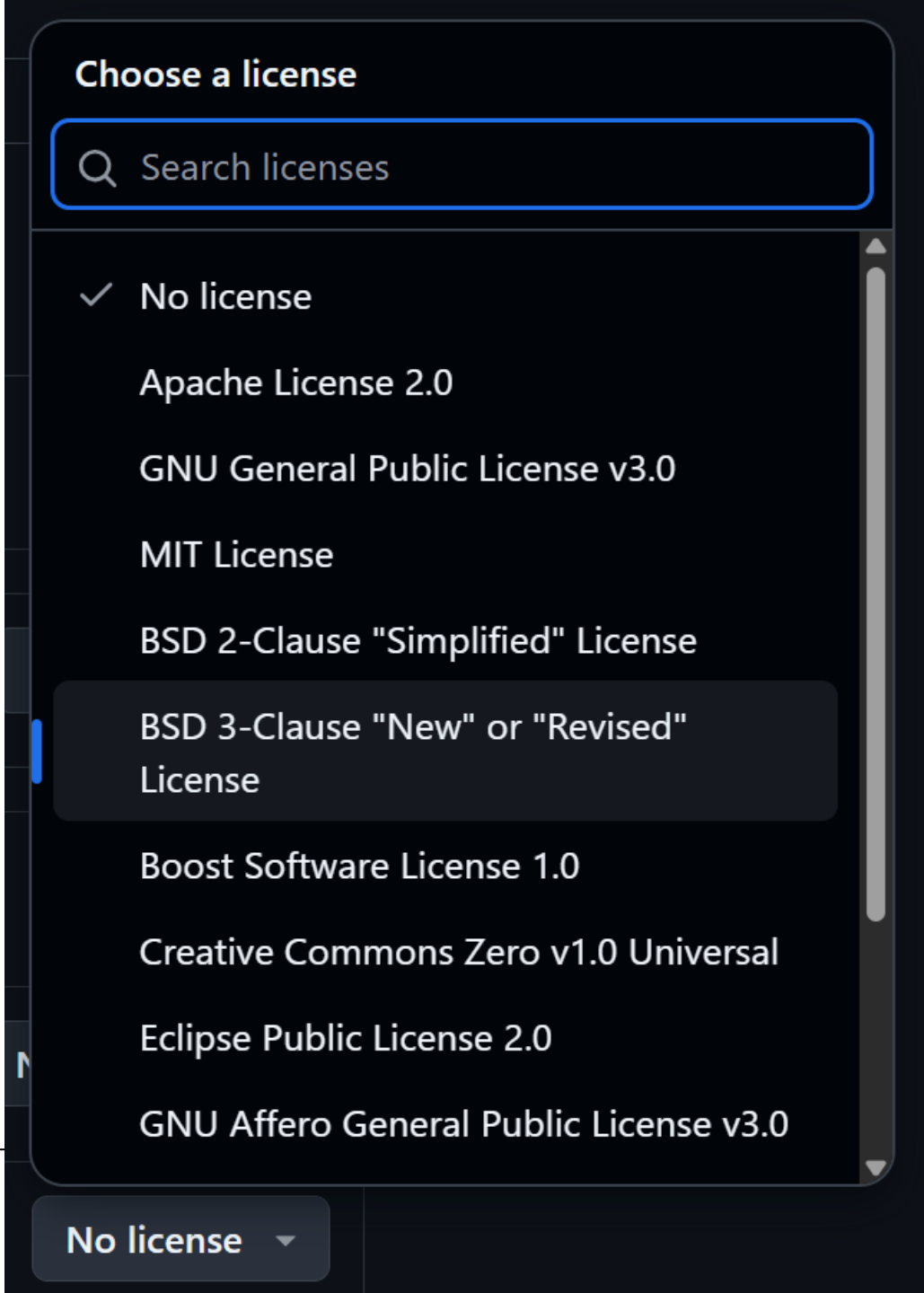
Example .gitignore

- CHAFF (Concealed, Hidden, And Forgotten Files)
- .DS_Store files are created automatically by MacOS computers and are not visible to those using that OS
- These files are used to keep track of metadata used by Finder–such as display options or icon positions–and Spotlight's search features

```

# DS store created on a mac
.DS_Store
._.DS_Store
**.DS_Store
**/._.DS_Store

# PowerPoint temporary
~$*.ppt*

```

Licenses

- A license is a legal permission that specifies what you are (and are not) allowed to do a work
- Applies to how you can use, modify, and distribute the work
- Apply to creative work, software, datasets, intellectual property, and more
- Consider how permissive you want to be
- For code, check out the website “Choose an open source license” https://choosealicense.com/
- Feel free to reach out and ask us
- Note: I am not a lawyer



choosealicense.com

- Has more licenses than the ones available by default on GitHub
- Use this option to add it to your GitHub repo

Home / Licenses

Creative Commons Attribution Share Alike 4.0 International

Copy license text to clipboard

Similar to CC-BY-4.0 but requires derivatives be distributed under the same or a similar, compatible license. Frequently used for media assets and educational materials. A previous version is the default license for Wikipedia and other Wikimedia projects. Not recommended for software.

Permissions	Conditions	Limitations
Commercial use	License and copyright notice	Liability
Distribution	Same license	Patent use
Modification	State changes	Trademark use
Private use		Warranty

```
Attribution-ShareAlike 4.0 International

=======================================================================

Creative Commons Corporation ("Creative Commons") is not a law firm and
does not provide legal services or legal advice. Distribution of
Creative Commons public licenses does not create a lawyer-client or
other relationship. Creative Commons makes its licenses and related
information available on an "as-is" basis. Creative Commons gives no
warranties regarding its licenses, any material licensed under their
terms and conditions, or any related information. Creative Commons
disclaims all liability for damages resulting from their use to the
fullest extent possible.

Using Creative Commons Public Licenses

Creative Commons public licenses provide a standard set of terms and
conditions that creators and other rights holders may use to share
original works of authorship and other material subject to copyright
and certain other rights specified in the public license below. The
following considerations are for informational purposes only, are not
exhaustive, and do not form part of our licenses.

     Considerations for licensors: Our public licenses are
     intended for use by those authorized to give the public
     permission to use material in ways otherwise restricted by
```

Suggest this license

Make a pull request to suggest this license for a project that is not licensed. Please be polite: see if a license has already been suggested, try to suggest a license fitting for the project's community, and keep your communication with project maintainers friendly.

Enter GitHub repository URL

How to apply this license

Create a text file (typically named LICENSE or LICENSE.txt) in the root of your source code and copy the text of the license into the file. It is also acceptable to solely supply a link to a copy of the license, usually to the canonical URL for the license.

Optional steps

Add `CC-BY-SA-4.0` to your project's package description, if applicable (e.g., Node.js, Ruby, and Rust). This will ensure the license is displayed in package directories.

Source

Who's using this license?

Flight rules for Git
Material Design Iconic Font
OWASP MSTG

midnitelibrary / RDM-Escape-Room

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

RDM-Escape-Room

Public

Pin | Watch 0 | Fork 0 | Star 0

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

midnitelibrary Create .gitignore 2338fc1 · now 2 Commits

.gitignore	Create .gitignore	now
LICENSE	Initial commit	1 minute ago
README.md	Initial commit	1 minute ago

README | CC0-1.0 license

RDM-Escape-Room

About

No description, website, or topics provided.

Readme
CC0-1.0 license
Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Help! I'm stuck, where do I go?

- **Trainings with Center for Research Data and Digital Scholarship (CRDDS)**: https://www.colorado.edu/crdds/
- **Software Carpentries tutorial:** https://swcarpentry.github.io/git-novice/index.html
- **GitHub Student Developer Pack:** https://education.github.com/pack
- **Helpdesk**: rc-help@colorado.edu

Survey and feedback

Survey: http://tinyurl.com/curc-survey18

Slides: https://github.com/ResearchComputing/git_github_in_depth_short_course